

RESPONSIBILITY MAPPING & ASSET DEFINITION

Name- Atharv Ambre

Prn-2124UCSM1056

Unit 1 – AWS 3-Tier Web Application

Project Title: AWS 3-Tier Web Application using Terraform

Cloud Provider: Amazon Web Services (AWS)

Infrastructure as Code: Terraform

AWS Region: Asia Pacific (Mumbai) – ap-south-1 **Architecture:**

Web Tier → Application Tier → Database Tier

GitHub Repository: <https://github.com/Atharv1056/CSA>

1. INTRODUCTION

The objective of this project is to design and deploy a secure three-tier web application architecture on Amazon Web Services (AWS) using Terraform Infrastructure as Code.

The application is divided into three logical tiers:

1. Web Tier
2. Application Tier
3. Database Tier

The Web Tier provides public access to the application. The Application Tier processes application requests and business logic, while the Database Tier securely stores application data.

The Web Tier is placed in a public subnet, while the Application and Database tiers are placed in private subnets. Security groups are used to control communication between the different tiers.

Terraform is used to create and manage the AWS infrastructure in a repeatable and automated manner.

2. ARCHITECTURE OVERVIEW

The architecture follows the communication flow: **Internet →**

Web Tier → Application Tier → Database Tier The AWS VPC

uses the CIDR block 10.0.0.0/16.

The network is divided into the following subnets:

- Web Public Subnet – 10.0.1.0/24
- Application Private Subnet – 10.0.2.0/24
- Database Private Subnet 1 – 10.0.3.0/24

- Database Private Subnet 2 – 10.0.4.0/24

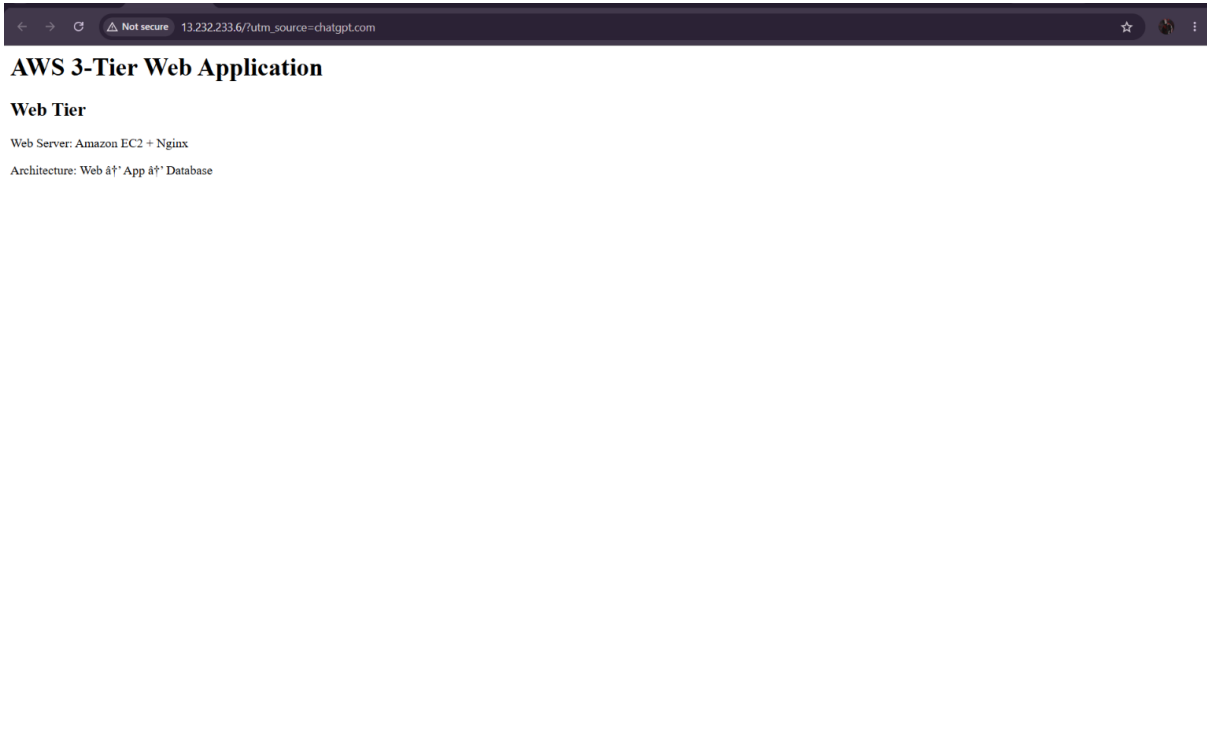


Figure 1: Successfully deployed AWS 3-Tier Web Application Web Tier running on Amazon EC2 with Nginx.

3. ASSET DEFINITION

The major AWS assets used in the project are listed below.

Asset	AWS Service	Purpose
VPC	Amazon VPC	Provides an isolated network environment
Web Server	Amazon EC2	Hosts the Nginx web server
Application Server	Amazon EC2	Handles application processing
Database	Amazon RDS	Stores application data using MySQL
Internet Gateway	Amazon VPC	Provides Internet connectivity to the public subnet
Web Security Group	EC2 Security Group	Controls Web Tier traffic
App Security Group	EC2 Security Group	Controls Application Tier traffic
DB Security Group	EC2 Security Group	Controls Database Tier traffic

Public Route Table	Amazon VPC	Provides routing for the Web subnet
Private Route Table	Amazon VPC	Provides routing for private subnets
DB Subnet Group	Amazon RDS	Defines the private subnets used by RDS

4. DEPLOYED AWS RESOURCES

The Terraform configuration successfully deployed **19 AWS resources**.

The deployment includes:

- 1 VPC
- 4 subnets
- 2 EC2 instances
- 1 Amazon RDS MySQL database
- 1 Internet Gateway
- Public and private route tables
- Route table associations
- 3 security groups
- 1 RDS subnet group

The Web Tier EC2 instance received the public IP address:

13.232.233.6

The Application Tier EC2 instance received the private IP address:

10.0.2.57

The Database Tier is hosted using Amazon RDS and is configured as **not publicly accessible**.

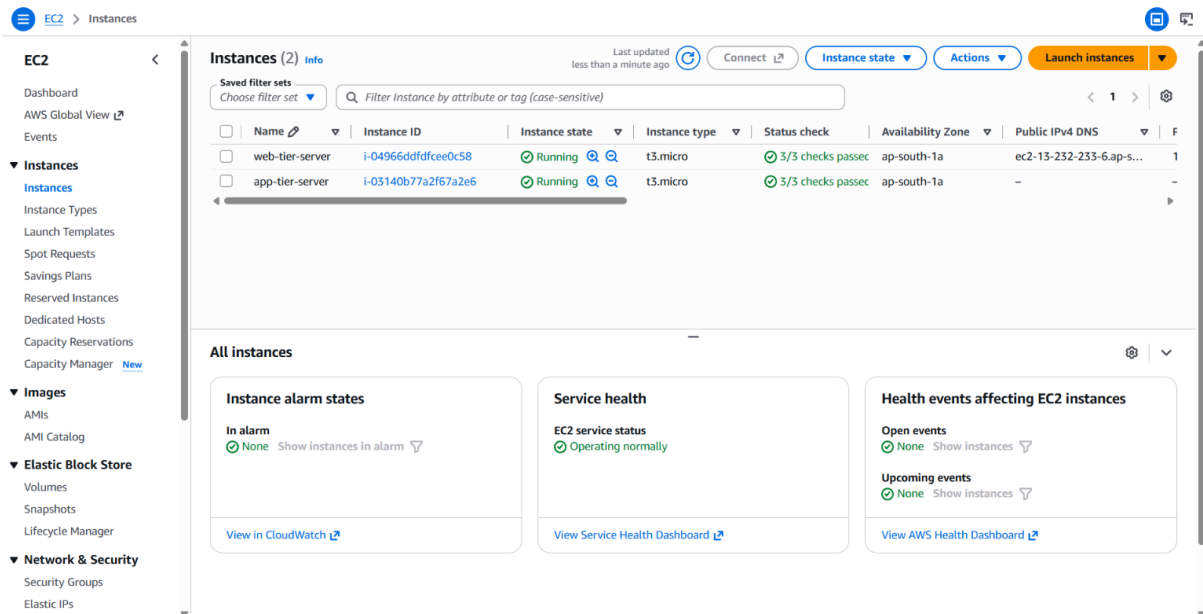


Figure 2: AWS EC2 Console showing the deployed Web Tier and Application Tier instances.

5. SECURITY GROUP CONFIGURATION

Separate security groups are used for the Web, Application, and Database tiers.

Web Tier Security Group

The Web Tier allows HTTP traffic on port 80.

SSH access on port 22 is also configured for administration during the academic lab.

Protocol	Port	Source	Purpose
TCP	80	0.0.0.0/0	HTTP Web Traffic
TCP	22	0.0.0.0/0	SSH Administration

Application Tier Security Group

The Application Tier allows TCP traffic on port 8080 only from the Web Tier Security Group.

Protocol	Port	Source	Purpose
TCP	8080	Web Tier SG	Application Traffic

Database Tier Security Group

The Database Tier allows MySQL traffic on port 3306 only from the Application Tier Security Group.

Protocol	Port	Source	Purpose
TCP	3306	App Tier SG	MySQL Database Traffic

Therefore, the communication flow is:

Internet → Web Tier → App Tier → Database Tier

The Application and Database tiers do not have direct public inbound access.

Security Note: SSH port 22 is included for this academic lab configuration. In a production environment, SSH should be restricted to trusted administrator IP addresses or replaced with a managed access mechanism.

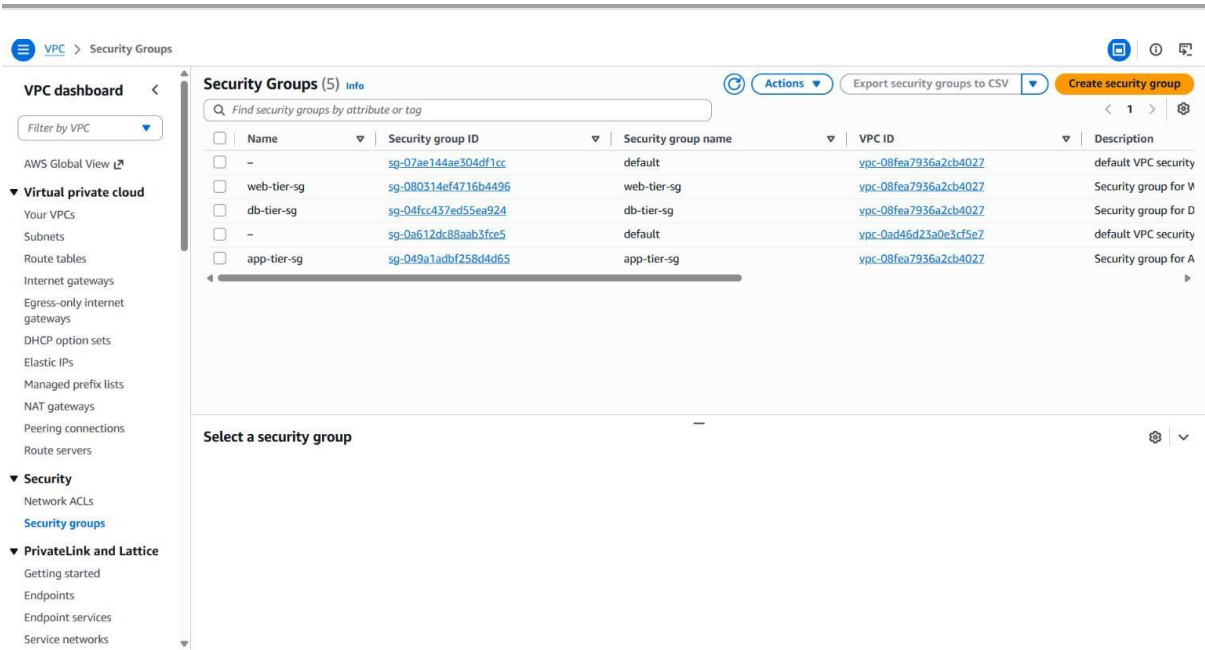


Figure 3: Security Groups controlling communication between the Web, Application, and Database tiers.

6. DATABASE TIER

The Database Tier uses Amazon RDS for MySQL.

Database Configuration

Database Engine: MySQL

Engine Version: 8.0

Instance Class: db.t3.micro

Allocated Storage: 20 GB

Storage Type: GP3

Database Name: studentdb

Database Port: 3306

Publicly Accessible: No

Subnet Placement: Private Database Subnets

The RDS database is protected by a dedicated Database Security Group. Database traffic is permitted only from the Application Tier Security Group. The database is therefore isolated from direct Internet access.

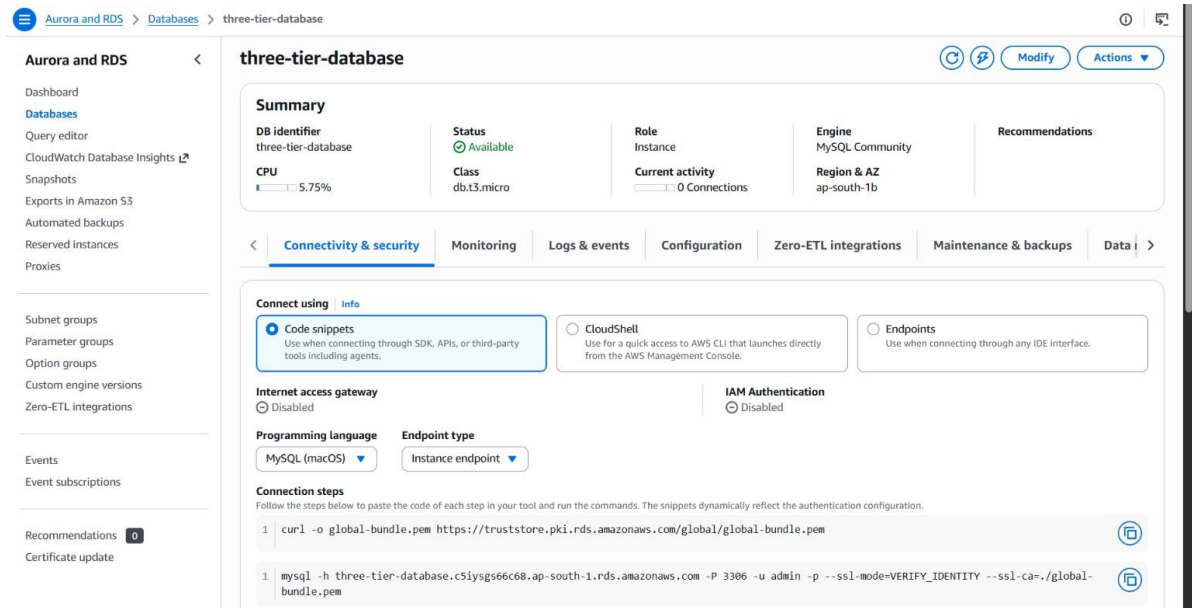


Figure 4: Amazon RDS MySQL database deployed in the private Database Tier.

7. AWS SHARED RESPONSIBILITY MODEL

AWS follows the Shared Responsibility Model.

AWS is responsible for security **of the cloud**, while the customer is responsible for security **in the cloud**. The exact responsibilities vary depending on the AWS service being used.

Security Control	AWS	Customer
Physical Data Centers	✓	
Physical Hardware	✓	
AWS Global Infrastructure	✓	
VPC Configuration		✓
Subnet Configuration		✓
Route Tables		✓
Security Groups		✓
EC2 Operating System		✓
EC2 Application Software		✓
EC2 Patching		✓
Application Data		✓
Database Credentials		✓
IAM Permissions		✓

RDS Underlying Infrastructure	✓	
RDS Configuration	Shared	✓
Encryption Configuration	Shared	✓
Monitoring Configuration	Shared	✓

8. TERRAFORM IMPLEMENTATION

The AWS infrastructure is managed using Terraform Infrastructure as Code.

Project Structure aws-3-tier-project/

```
|
|— main.tf
|— ec2.tf
|— rds.tf
|— security_groups.tf
|— variables.tf
|— outputs.tf
|— README.md
|— .gitignore
└─ .terraform.lock.hcl Terraform
```

File Description main.tf

Defines the VPC, subnets, Internet Gateway and route tables.

ec2.tf

Defines the Web and Application EC2 instances.

rds.tf

Defines the RDS subnet group and MySQL database.

security_groups.tf

Defines security groups and their inbound and outbound rules.

variables.tf

Defines configurable Terraform variables such as AWS region and database password.

outputs.tf

Provides useful deployment outputs such as EC2 IP addresses and database endpoint.

README.md

Contains project documentation and architecture information.

.gitignore

Prevents Terraform state files and local variable files from being uploaded to GitHub.

9. DEPLOYMENT VERIFICATION

The Terraform configuration was successfully deployed to AWS.

Terraform reported:

Apply complete! Resources: 19 added, 0 changed, 0 destroyed. The Web

Tier was successfully accessed through its public IP address:

<http://13.232.233.6>

The Web Tier successfully displayed the custom Nginx webpage, confirming that the EC2 instance and Nginx web server were operational.

The Application Tier is deployed inside a private subnet with the private IP address:

10.0.2.57

The Database Tier is deployed using Amazon RDS MySQL and is configured as not publicly accessible.

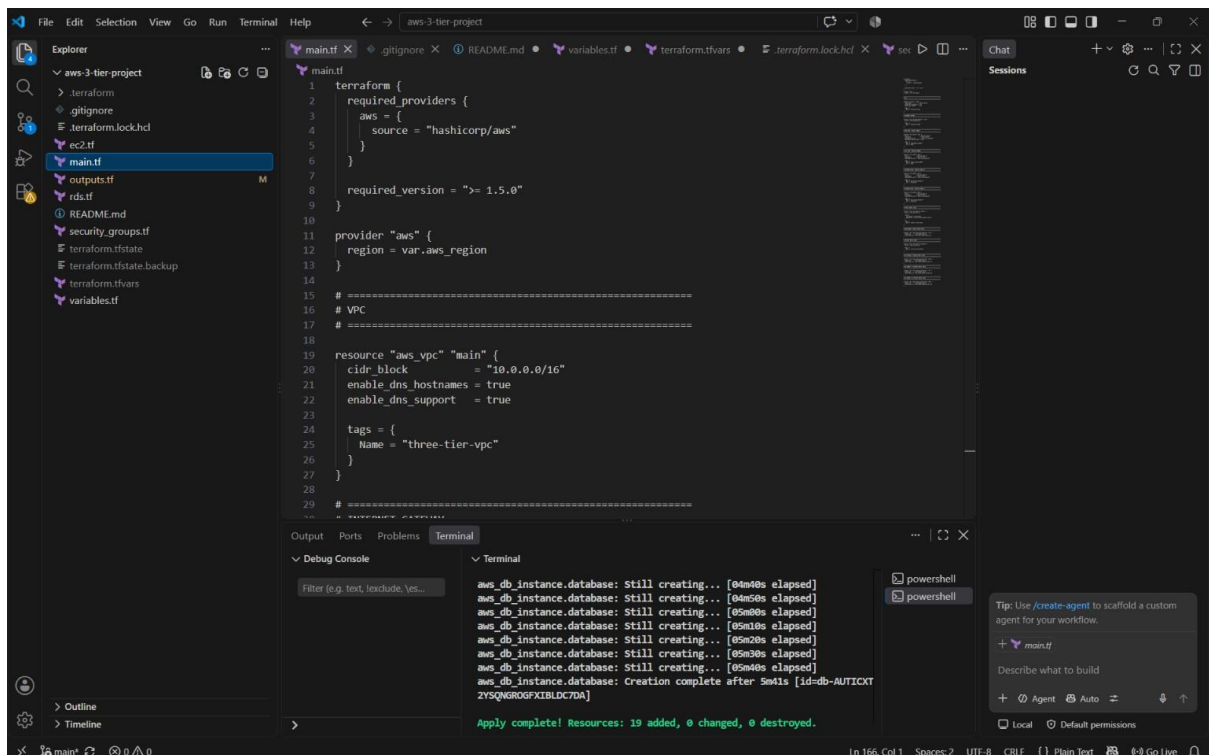


Figure 5: Terraform successfully deploying the AWS 3-tier infrastructure.

10. SECURITY CONSIDERATIONS

The following security measures have been implemented:

- The Database Tier is not publicly accessible.
- The Application Tier is placed in a private subnet.
- Database traffic is restricted to the Application Tier.
- Application traffic is restricted to the Web Tier.
- Separate security groups are used for each tier.
- The Web, Application and Database resources are logically separated.
- Database credentials are supplied through a Terraform variable instead of being hardcoded in the main RDS configuration.
- Terraform .tfvars files and state files are excluded from GitHub using .gitignore.
- The architecture follows the AWS Shared Responsibility Model.

11. CONCLUSION

This project successfully demonstrates the design and deployment of a secure three-tier web application architecture using Amazon Web Services and Terraform.

The Web Tier provides controlled public access through Amazon EC2 and Nginx. The Application Tier is isolated inside a private subnet and accepts application traffic from the Web Tier. The Database Tier uses Amazon RDS MySQL and remains private, allowing database communication only from the Application Tier.

Security groups provide controlled communication between the different tiers, while the AWS Shared Responsibility Model identifies the security responsibilities of AWS and the customer.

Terraform provides an Infrastructure as Code approach that makes the architecture repeatable, organized, and easier to manage.

The infrastructure has been successfully deployed and verified on AWS.

12. GITHUB REPOSITORY

Project Source Code

GitHub Repository: <https://github.com/Atharv1056/CSA>

The repository contains the Terraform source code, configuration files, project documentation, and supporting files required to reproduce the AWS 3-tier architecture.